



BACKEND



1

Learning Outcomes

By the end of this lesson, students will be able to:

- Understand the role of backend in a web application.
- Build a server using Node.JS and Express.JS
- Build RESTful APIs using Node.JS and Express.JS
- Apply clean architecture and best practices using MVC and Service Layer structure.
- Work with MongoDB and Mongoose to model and manage data.

2



1. Introduction

3

Questions

- What can't Frontend do?
 - Access database directly
 - Perform secure logic
 - Manage users & permissions
- Why do we need a backend?
 - Communication between client and database
 - Data storage
 - Business logic
 - Authentication & authorization
 - Result returned: JSON responses (REST API)

4

Backend Technology Landscape

- Languages: JavaScript, TypeScript, Python, Go, Java
- Frameworks:
 - Node.js + Express.js (common)
 - NestJS (enterprise)
 - Fastify (performance)
- Databases:
 - MongoDB (NoSQL)
 - PostgreSQL / MySQL (SQL)
- Infrastructure:
 - Docker, CI/CD, Cloud, Serverless

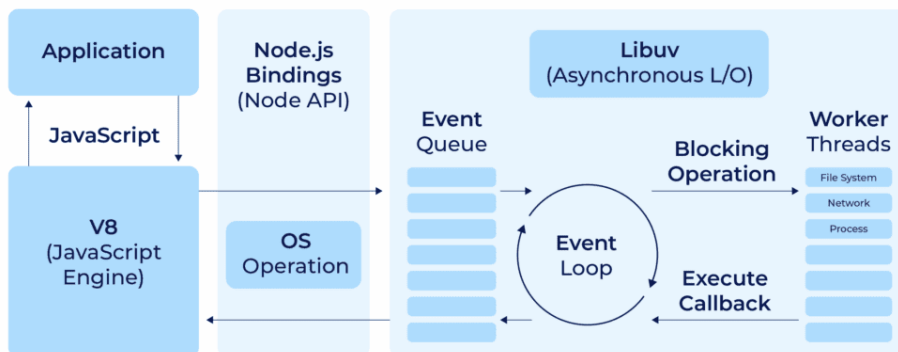
Why JavaScript for Backend?

- One language for the entire stack
 - Use JavaScript on both Frontend and Backend
 - Easier learning and unified workflow for teams
- Fast and efficient
 - Built on Google's V8 Engine
 - Non-blocking handles thousands of requests concurrently
- Huge ecosystem (NPM)
 - Large package ecosystem
 - Libraries for everything: APIs, auth, databases, testing..
 - Strong community support
- Modern backend-friendly features
 - Async/Await
 - TypeScript compatibility
 - Microservices & Serverless support

What is Node.js?

- Node.js is a JavaScript runtime environment
 - Allows JavaScript to run outside the browser
 - Built on Google Chrome's V8 JavaScript engine
 - Designed for building fast, scalable, network applications
- Key characteristics
 - Event-driven architecture
 - Non-blocking I/O model
- Common use cases
 - Realtime applications (chat, notifications)
 - Microservices architecture
 - Backend for web & mobile apps

Node.js Architecture



<https://litslink.com/blog/node-js-architecture-from-a-to-z>

What is Express.JS?

- A **web framework** built on top of Node.JS,
- Provides tools to build HTTP servers, REST APIs, and web applications.
- Makes server development easier and cleaner.
- Features
 - Simple & Minimal: focuses on the essentials: routing, middleware, request & response.
 - Flexible architecture: choose patterns (MVC, services, etc.).
 - Huge ecosystem: thousands of packages for authentication, logging, security, rate limiting, validation, etc.
 - Industry adoption: used by Uber, PayPal, IBM.



9

Basic Routing in Express

- Routing = mapping URLs to functions.
- `app.METHOD(PATH, HANDLER)`
- Where:
 - METHOD → HTTP verb (GET, POST, PUT, DELETE)
 - PATH → URL route (/, /users, /products/:id)
 - HANDLER → function (req, res) => { ... }

- Example

```
app.post('/users', (req, res) => {
    res.send('User created');
});
```



10

HTTP Methods (GET, POST, PUT, DELETE)

- HTTP methods define **the type of action** the client wants to perform on the server.
- They are the foundation of **RESTful API design**.

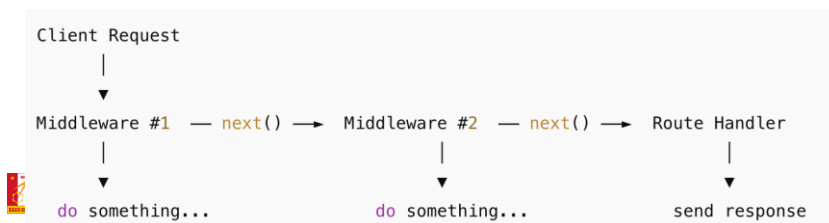
Method	Purpose	Typical Use
GET	Read data	Fetch resources
POST	Create data	Add new resource
PUT	Update data (replace)	Update full resource
DELETE	Remove data	Delete resource

What is Middleware?

- Middleware = steps the request must pass through.

```
function middleware(req, res, next) {
  // do something...
  next(); // pass control to the next middleware
}
```

```
app.use((req, res, next) => {
  console.log('Time:', Date.now());
  next(); // without this, request will stop here
});
```



MVC Architecture

- **MVC (Model – View – Controller)** is a design pattern that separates an application into 3 clear layers:
 - **Model** → Data & database logic
 - **View** → Display (HTML, JSON)
 - **Controller** → Request handling & business logic
- Flow (RESTful)
 - Client→Controller→Model→Database→Model→Controller→Response (JSON)



13

Service Layer: Avoiding Fat Controller

- **Service Layer** contains: Business logic & Data processing

Controller

```
export const createUser = async (req, res, next) => {
  const result = await userService.createUser(req.body);
  res.status(201).json(result);
};
```

Model

```
UserModel.create(data);
```

Service

```
export const userService = {
  async createUser(data) {
    // business rules
    if (data.age < 18) throw new Error('Min age is 18');

    // call DB
    return UserModel.create(data);
  }
};
```



14

Backend Folder Structure

- config: system-level configuration
- routes: API endpoints
- controllers: request/response handling, no business logic
- services: business logics
- repositories: data access layer, handle queries to database
- models: database schema
- validations: request validation
- middlewares: reusable request functions
- utils: shared helper functions
- libs: external service integrations
- constants: store constant values
- app.js: initializes express app
- server.js: starts the server

```
src/
├─ config/
├─ routes/
├─ controllers/
├─ services/
├─ repositories/
├─ models/
├─ validations/
├─ middlewares/
├─ utils/
├─ libs/
├─ constants/
├─ app.js
└─ server.js
```



TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

15



4. MongoDB and Mongoose

16

SQL vs NoSQL

- SQL (Relational Databases)
 - Examples: MySQL, PostgreSQL, SQL Server, Oracle
 - Structured, table-based data
- NoSQL (Non-Relational Databases)
 - Examples: MongoDB, Redis, Cassandra, Neo4j
 - Flexible schema
- Key Differences
 - Schema: Fixed (SQL) vs Flexible (NoSQL)
 - Structure: Tables vs Documents/Key-Value/Graphs
 - Use Cases: Complex relations (SQL) vs High volume & dynamic data (NoSQL)



17

MongoDB Atlas

- Cloud database service for MongoDB
- Key Features
 - Automated Scaling
 - Global Clusters
 - Built-in Security:
 - Automated Backups with point-in-time recovery
 - Performance Monitoring dashboards
- Flow: Create Cluster → Get Connection String → Connect via Driver/Mongoose → Build App
 - Cluster = a MongoDB database environment
 - Contains many databases
 - Ensures performance, scaling, security, etc.



18

What is Mongoose?

- An ODM (Object Data Modeling) library for MongoDB
 - Provides a structured way to interact with MongoDB
 - Mongoose adds **structure, validation, and consistency** to MongoDB collections.

```
const UserSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number
});
```

```
age: { type: Number, min: 18 }
```



19

Mongoose Schema

- A Mongoose Schema defines the structure and data types of documents in a MongoDB collection.
- Common Data Types
 - String – for text fields (name, email, title, etc.)
 - Number – for integers, floats (age, price, score)
 - Boolean – for true/false values (isActive, isAdmin)
 - Date – for timestamps and calendar dates (createdAt, birthday)

```
const mongoose = require("mongoose");

const UserSchema = new mongoose.Schema({
  name: { type: String, required: true },
  age: { type: Number, min: 0 },
  isAdmin: { type: Boolean, default: false },
  createdAt: { type: Date, default: Date.now }
});
```



20

Model (mongoose.model)

- Models represent collections in MongoDB
- Function: `mongoose.model(modelName, schema).`

```
// Create Model
```

```
const User = mongoose.model("User", UserSchema);
```

- **"User"**: Model name in Mongoose
- Mongoose creates **users** collection in MongoDB
- **User**: JS variable
 - `User.create({ name: "Minh" });`
 - `User.find();`
 - `User.findById(id);`

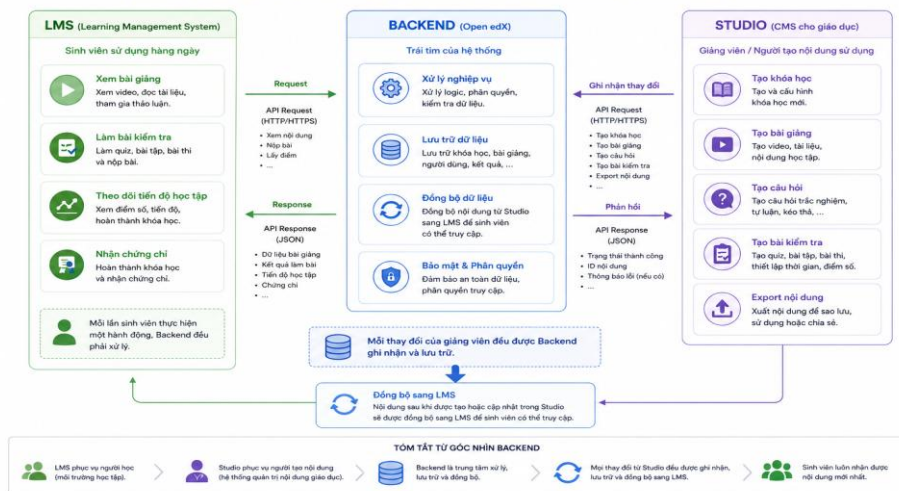


21

Case Study 1

LMS & STUDIO – HAI GIAO DIỆN, MỘT HỆ THỐNG

LMS phục vụ người học • Studio phục vụ người tạo nội dung • Backend kết nối và đồng bộ mọi thứ

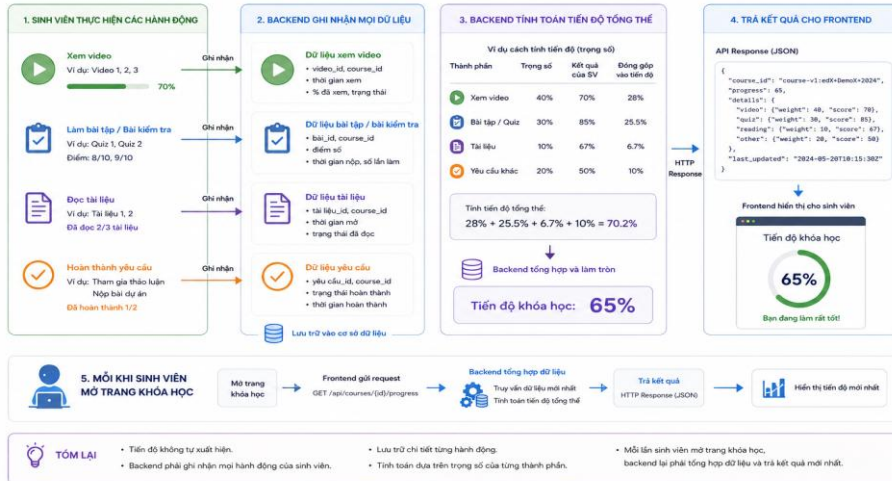


22

Case Study 2

LÀM THẾ NÀO HỆ THỐNG BIẾT SINH VIÊN ĐÃ HOÀN THÀNH 65% KHÓA HỌC?

Đây không phải là điều tự nhiên xuất hiện — Backend phải ghi nhận, tính toán và tổng hợp dữ liệu.



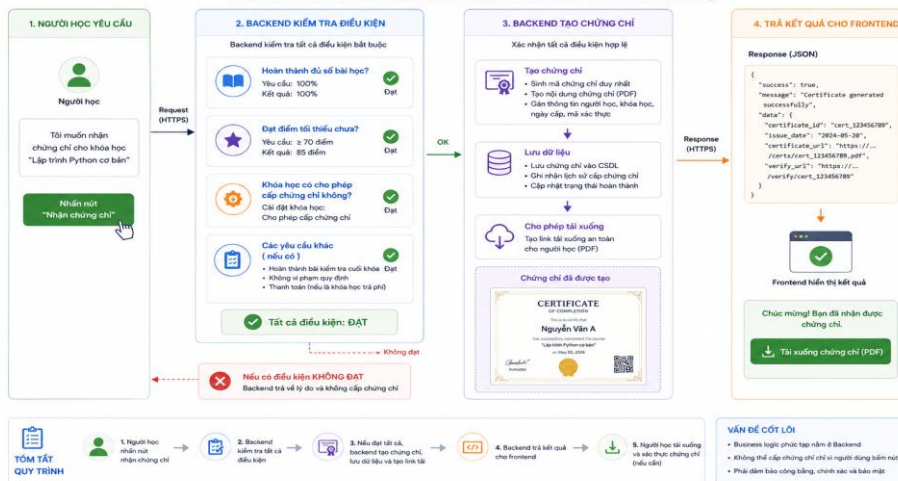
23

Case Study 3

QUY TRÌNH CẤP CHỨNG CHỈ – VÍ DỤ ĐIỂN HÌNH VỀ BUSINESS LOGIC

Backend không thể cấp chứng chỉ chỉ vì người dùng bấm nút.

Nó phải kiểm tra rất nhiều điều kiện, xác nhận, sau đó mới tạo và cho phép tải xuống.



24

